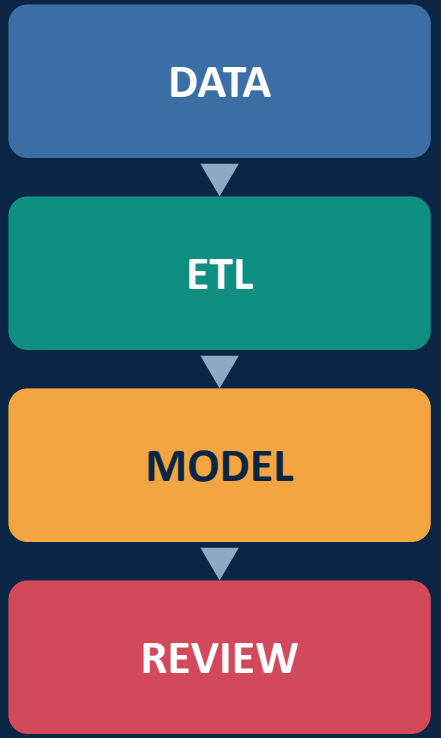
CPP 4201 | ADVANCED PYTHON PROGRAMMING | ASSIGNMENT 2

Predicting University Dropout

A reproducible ETL and machine-learning pipeline, built with advanced Python



Tony Kenga 24/03652
Baraka Blessings 24/04272
Linet Maina 24/00632

KCA University | BSc Data Science | October 2026

Problem, objectives and research questions

Problem: estimate a student's recorded outcome (Dropout, Enrolled or Graduate) from 36 features, using a pipeline that anyone can re-run and audit.

RESEARCH QUESTIONS

- **RQ1** How well can the features predict the outcome?
- **RQ2** Which features matter most to the model?
- **RQ3** How can rule-based intent routing give local, practical signposting?
- **RQ4** How do advanced Python techniques improve reliability?

OBJECTIVES

Obtain and validate

Official UCI data, schema and value checks

Prepare features

Nominal codes vs numeric measures

Compare 3 classifiers

Same split, folds and metric

Evaluate and explain

Metrics, confusion matrix, importance

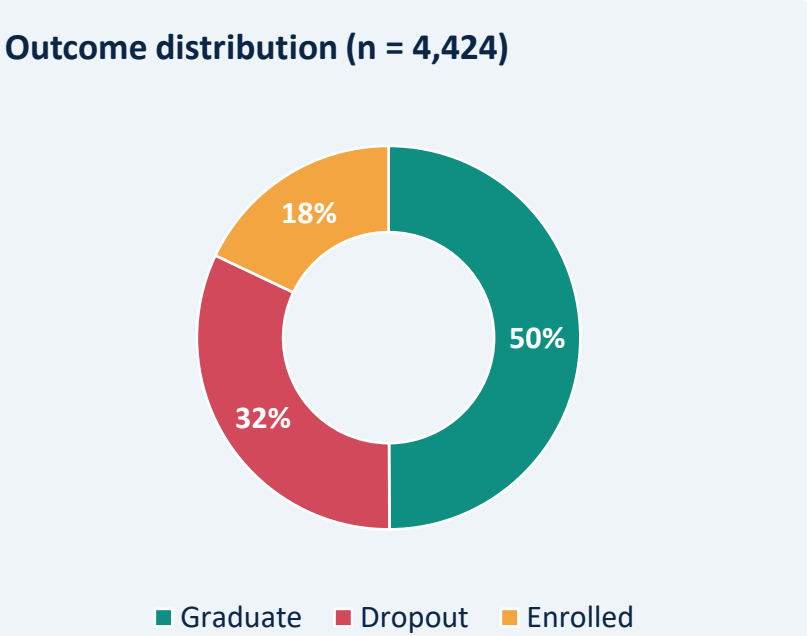
The data: clean, but imbalanced

4,424
student records

36
predictors + 1 target

0
missing cells or duplicates

3
outcome classes



Source: UCI ML Repository, dataset 697 (CC BY 4.0), Portuguese higher-education cohort. ZIP and CSV verified with SHA-256.

Majority-class guess = 49.9% accuracy, so macro-F1 is our primary metric.

System architecture: modular, one job per module

01

Extract

`extract.py`

Download ZIP, read data.csv safely

02

Validate

`validate.py`

Schema, widths, labels, missingness

03

Transform

`transform.py`

Split, one-hot and scale by column type

04

Train

`model.py`

3 pipelines, 5-fold stratified CV

05

Evaluate

`evaluate.py`

Holdout metrics and importance

06

Serve

`support_app.py`

Streamlit views, saved artifacts

Separate track: Kenyan campus Student Support hub

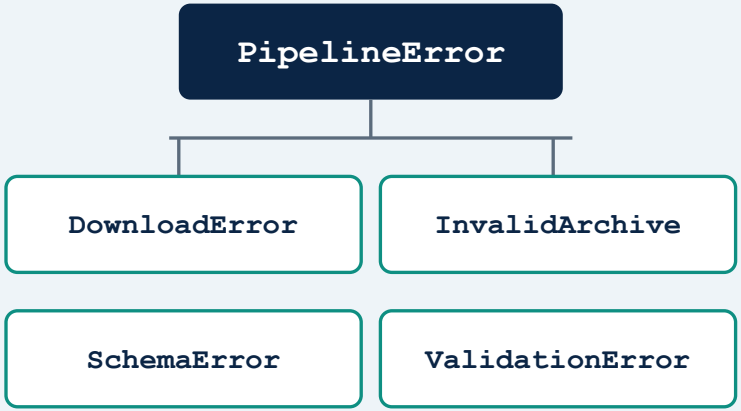
Rule-based intent chat (CATs, HELB, fees), study reset, verified resource directory and pseudonymous case follow-up. It never calls the dropout model.

ETL and validation: fail early, fail clearly

WHAT THE VALIDATOR CHECKS

- 37 unique columns, target in the expected place
- Row width on every line, read lazily
- Target is Dropout, Enrolled or Graduate
- No missing cells; duplicates logged, never dropped
- All predictors numeric
- SHA-256 digest of ZIP and CSV, read in chunks

DOMAIN ERRORS, NOT GENERIC ONES



bounded_retry: up to 3 attempts with short delays, network errors only. A malformed dataset is never retried.

Modelling protocol: three candidates, one fair test

Training: 3,539 rows (80%), 5-fold stratified CV | **Test: 885 (20%)**

Logistic Regression

Linear baseline

class_weight balanced, 2,000 iterations

Random Forest

Bagged trees

300 trees, min leaf 2, class_weight balanced

Gradient Boosting

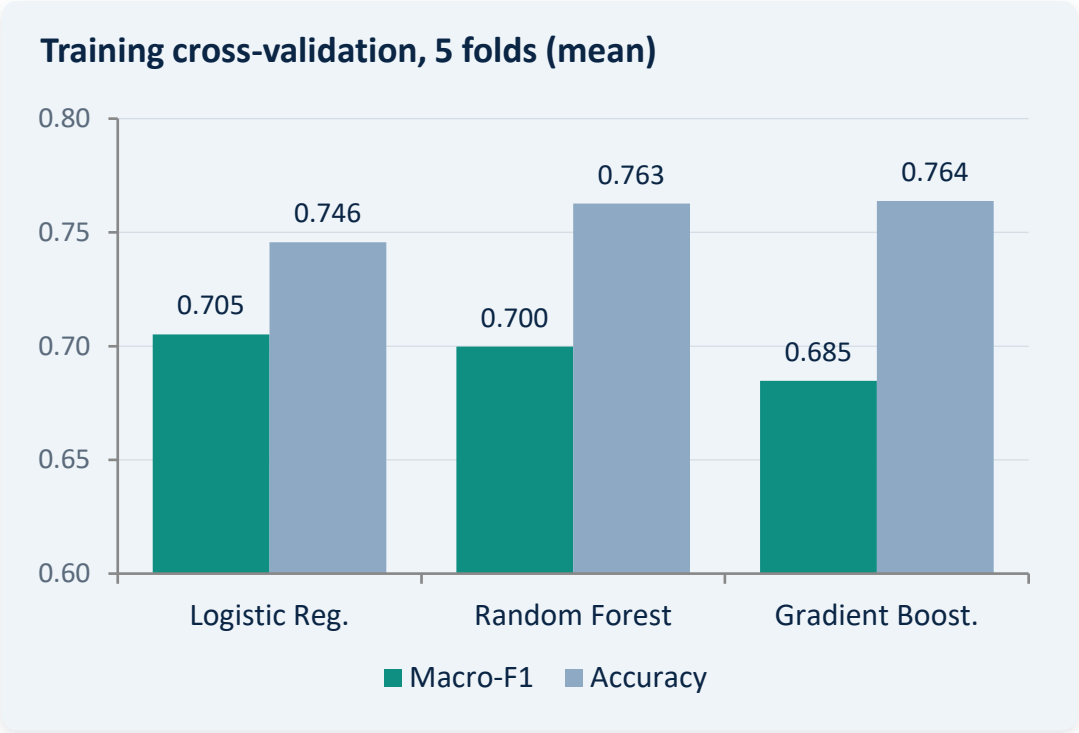
Boosted trees

120 estimators, learning rate 0.08, depth 2

Leakage-safe by design

ColumnTransformer + estimator live in one scikit-learn Pipeline, so one-hot encoding (nominal codes) and scaling (numeric measures) are fitted inside each fold. The best mean CV macro-F1 wins and touches the test set once.

Results: Logistic Regression wins on macro-F1



0.7155

held-out macro-F1 (selected model)

0.7559

held-out accuracy, 885 students

Trees score higher accuracy but lower macro-F1: they serve the Enrolled class less evenly. One split, one dataset: not a guarantee elsewhere.

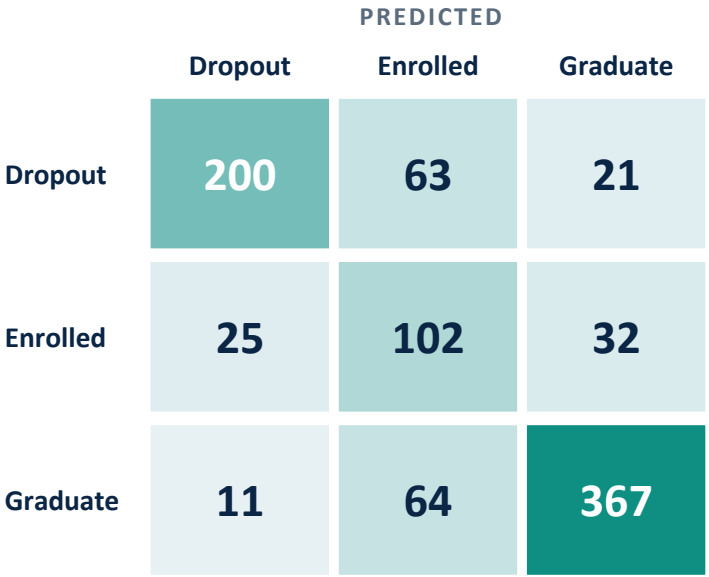
Where the model works, and where it struggles

Class	Precision	Recall	F1	Support
Dropout	0.847	0.704	0.769	284
Enrolled	**0.445**	0.642	**0.526**	159
Graduate	0.874	0.830	0.852	442

Enrolled is the weak class (F1 0.526): many Enrolled predictions were really another outcome.

Top features: approved units, 2nd sem (drop 0.203) and 1st sem (0.138). Late, correlational signals, not causes.

Confusion matrix (held-out, n = 885)



Rows = actual outcome, columns = predicted

Advanced Python: evidence in the code

Protocol + type hints

validate.py

```
class FrameValidator(
    Protocol):
  def validate(...): ...
```

Dataclass + StrEnum

config.py, inference.py

```
@dataclass(frozen=True,
  slots=True)
class PipelineConfig: ...
```

Context managers

extract.py

```
with urlopen(url) as r:
    data = r.read()
```

Decorator + ParamSpec

extract.py

```
@bounded_retry(attempts=3)
def download_dataset(...)
```

Generator

extract.py

```
def iter_nonempty_lines(...):
    yield number, line
```

match / case

evaluate.py

```
match label:
    case "Dropout": ...
```

Also: logging, a custom exception hierarchy, 34 passing pytest tests, Ruff and mypy checks.

Conclusions, limits and next steps

WHAT WE FOUND

RQ1 Macro-F1 0.716, accuracy 0.756 on 885 held-out students; Enrolled is weakest

RQ2 Approved units in semesters 1 and 2 dominate importance

RQ3 Transparent rules route CAT, HELB and fee stress to human referrals

RQ4 Protocols, generators, context managers, decorators and tests make runs traceable

Thank you. Questions?

LIMITS

- Portuguese cohort, not validated for Kenya
- Random split, no subgroup or fairness tests
- Prediction is not cause; humans decide

BEFORE ANY KENYAN USE

- Lawful, local data
- Fairness and privacy review
- Co-design with campus teams